



Preparation for Cumulative Final Exam on Friday 05.16 at 1:30 PM

Completion requirements

This section reviews material introduced after Test 6. These topics will appear on the final exam but were not assessed on the earlier tests.

Use the review guides, past tests, readings, slides, and labs to prepare. There's more content here than you can master in a single day, so focus your energy where your understanding is weakest. Good luck!

Classical Language Models and Naive Bayes

- Naive Bayes is a simple but powerful **probabilistic classifier** often used in NLP tasks like spam filtering or document classification.
- It applies **Bayes' Theorem** to estimate which class label is most likely, assuming that all features are **independent** given the label (this is why it's called "naive").
- You don't need to memorize formulas, but you should understand the basic idea: the model uses past data to make predictions about new examples.
- It's fast and effective, especially for simple classification tasks—but struggles when features are correlated or continuous. (We skipped topics like smoothing and log-space math.)

Be able to **define or use in a sentence**:

- Bag of Words
- N-grams (Unigram, Bigram, Trigram)
- Spam classification ("Spam" vs "Ham")
- Bayes' Theorem

Understand the **concepts** behind:

- Why Naive Bayes is considered "naive"
- How prior data informs classification
- When Naive Bayes works well—and where it falls short

Bayes' Theorem (Reference)

$$P(A|B) = P(B|A) * P(A) / P(B)$$

Plain English: It tells us how to update our belief about something (A) after seeing new evidence (B). In classification, it helps estimate which label is most likely based on observed features.

Large Language Models (LLMs)

This topic was introduced during our single-day unit on **Transformer-based language models** like GPT. Focus on basic ideas and definitions rather than technical depth.

Be able to **define or use in a sentence**:

- GPT (generative pretrained transformer)
- Generative models
- Pretrained models
- Transformer architecture
- Attention mechanism
- Self-attention
- Multi-headed attention
- Positional encoding
- Tokens and tokenization
- Morpheme (vs. phoneme)
- Semantic embedding
- Softmax (activation function)
- Temperature (in text generation)
- Top-k sampling
- Top-p sampling (nucleus sampling)
- Fine-tuning
- Reinforcement Learning from Human Feedback (RLHF)
- Context window
- Hallucination

Understand the **concepts** behind:

- The difference between **words**, **tokens**, **morphemes**, and **phonemes**
- How **tokenization** affects what a model "sees" (e.g., "walking!" → "walk", "ing", "!")
- How the **context window** limits model memory
- Why **multi-headed attention** matters: each head can attend to different parts of the input
- What **softmax** does: turns raw scores into probabilities
- How **temperature** adjusts randomness in text generation
- Why LLMs can **hallucinate** (confidently produce false information)
- Ethical risks: **biases** in training data can be reflected or amplified by the model

Embeddings Refresher

Embeddings are vectors—ordered lists of numbers—that capture word meaning. You can manipulate them like geometric vectors:

- Addition: $[1, 2, 3] + [4, 0, -1] = [5, 2, 2]$

- Subtraction: $[1, 2, 3] - [4, 0, -1] = [-3, 2, 4]$
- Dot product: $[1, 2, 3] \cdot [4, 0, -1] = 1 \times 4 + 2 \times 0 + 3 \times (-1) = 1$

Communicate and Operationalize

- Review the **analytics team roles** and **data analytics lifecycle** from earlier in the course.
- Know the four key deliverables at the end of a project:
 - Presentation for project sponsors
 - Presentation for analysts
 - Code
 - Technical specifications
- Consider how a presentation to sponsors differs from one to analysts. (DSC 500 Capstone will go deeper into these skills.)

Last modified: Tuesday, May 13, 2025, 3:26 PM